Numeric Literals in Java

There are four ways to represent integer numbers in the Java language: decimal (base 10), octal (base 8), hexadecimal (base 16), and from Java 7, binary (base 2).

One more new feature introduced in Java 7 was **numeric literals with underscores (_) characters**. This was introduced to increase readability. See below:

```
int pre7 = 1000000;     // pre Java 7 - we hope it's a million
int with7 = 1_000_000;  // much clearer!
```

Adjacent underscores ARE allowed.

But you must keep in mind the below gotchas:

```
int i1 = _1_000_000; // illegal, can't begin with an "_"
int i2 = 10_0000_0;  // legal, but confusing
```

NOTE: You can use the underscore character for any of the numeric types (including doubles and floats), but for doubles and floats, you CANNOT add an underscore character directly next to the decimal point.

Using a Base (Radix) other than 10

All four integer literals (binary, octal, decimal, and hexadecimal) are defined as `int` by default, but they may also be specified as `long` by placing a suffix of `L` or `l` after the number:

```
long jo = 110599L;
long so = 0xFFFFl;  // Note the lowercase 'l'
```

```
0 1 2 3 4 5 6 7 8 9 a b c d e f
```

Java accepts uppercase or lowercase letters for the extra digits *(one of the few places Java is not case-sensitive)*. You represent an integer in hexadecimal form by placing a `0x` in front of the number, as follows:

```
int x = 0X0001;     // equals to decimal 1
int y = 0x7fffffff; // equals to decimal 2147483647
int z = 0xDeadCafe; // equals to decimal -559035650
```

Binary Literals

From Java 7, you can initialize variables holding binary literals. But they must start with either `0B` or `0b`, as shown below:

```
int b1 = 0B101010;   // set b1 to binary 101010 (decimal 42)
int b2 = 0b00011;    // set b2 to binary 11 (decimal 3)
```

Octal Literals

Octal integers use only the digits 0 to 7. They have a radix of 8. In Java, you represent an integer in octal form by placing a zero in front of the number, as follows:

```
int six = 06;     // equal to decimal 6
int seven = 07;   // equal to decimal 7
int eight = 010;  // equal to decimal 8
int nine = 011;   // equal to decimal 9
```

Floating-point literals are type double by default, but you can't store a double value into a float variable, so create the value as a float:

`float x = 3.14f;  out.print(x);  → 3.14`